PROGRAM:

```
def encode(d):
    h = [0]*7
    h[2] = d[0]
    h[4] = d[1]
    h[5] = d[2]
    h[6] = d[3]
    h[0] = h[2] ^ h[4] ^ h[6]
    h[1] = h[2] ^ h[5] ^ h[6]
    h[3] = h[4] ^ h[5] ^ h[6]

def detect_and_correct(h):
    c1 = h[0] ^ h[2] ^ h[4] ^ h[6]
    c2 = h[1] ^ h[2] ^ h[5] ^ h[6]
    c4 = h[3] ^ h[4] ^ h[5] ^ h[6]
    error_pos = c1*1 + c2*2 + c4*4
    if error_pos != 0:
        print("error detected at bit position: {error_pos} -> Correcting...")
        h[error_pos-1] ^= 1
    else:
        print("No error detected.")
    return error_pos

def to_str(bits):
    return "".join(str(b) for b in bits)

def main():
    print("HAMMING CODE - ERROR DETECTION AND CORRECTION ==\n")
    n = int(input("Enter number of 4-bit data blocks in the stream: "))
```

Ex.No:6 ERROR DETECTION and CORRECTION USING HAMMING CODE

AIM:

To write a program to implement error detection & correction using the Hamming code

Algorithm:

- Start the program
- Read the no. of 4-bit data blocks (n) forming the I/P data stream
- ~~Read~~ for each 4bit data block D = d1 d2 d3 d4 perform hamming encoding.

a) Arrange 7 bits with parity bits at position 1,2,4 and data bits at positions 3,5,6 & 7

b) Compute the parity bits using even parity:

P1 = d1 XOR d2 XOR d4 (Covers bits positions 1,3,5,7)

P2 = d1 XOR d3 XOR d4 (Covers bit position 2,3,6,7)

P4 = d2 XOR d3 XOR d4 (Covers bit position 4,5,6,7)

c) Form the 7 bit Hamming code word: P1 P2 d1 P4 d2 d3 d4

- Simulate transmission by introducing a single bit error at a chosen bit position (entered by user) in the code word.

(2) At the receiver, recompute the check ~~bit~~ bits (syndrome) from the received code word:

```
for b in range(1, n+1):
    print(f"\n--Block {b}---")
    d = list(map(int, input("Enter 4 data bits (space seperated, eg: 1 0 1 1): ").split()))
    h = encode(d)
    print(f"original Data Bits : {to_str(d)}")
    print(f"Encoded Hamming Code (7 bits): {to_str(h)}")
    pos = int(input("Enter: "))
    if 1<= pos <= 7:
        h[pos-1] ^= 1
        print(f"Transmitted Code (with error induced at pos {pos}): {to_str(h)}")
    else:
        print(f"Transmitted code (no error induced): {to_str(h)}")
    print(f"Received Code : {to_str(h)}")
    print("Checking for errors....")
    detect_and_correct(h)
    print(f"corrected code : {to_str(h)}")
    rd = [h[2], h[4], h[5], h[6]]
    print(f"Recovered Data Bits: {to_str(rd)}")

if __name__ == "__main__":
    main()
```

$C_1 = P_1 \text{ XOR } d_1 \text{ XOR } d_2 \text{ XOR } d_4$

$C_2 = P_2 \text{ XOR } d_1 \text{ XOR } d_3 \text{ XOR } d_4$

$C_3 = P_4 \text{ XOR } d_2 \text{ XOR } d_3 \text{ XOR } d_4$

6. Compute error position = $C_4 C_2 C_1$ (binary no formed from the check bits).

7. If error position = 0, no error has occured
   Else the bit at "Error Position" is inverted (complicated) to correct the error

8. Extract the original 4 data bits from the corrected 7-bit code word

9. Repeat steps 3 to 8 for all n blocks in data stream

OUTPUT:

```
==HAMMING CODE - ERROR DETECTION AND CORRECTION==
Enter number of 4-bit data blocks in the stream: 2

--Block 1---
Enter 4 data bits (space seperated, eg: 1 0 1 1): 1 0 1 1
Original Data Bits : 1 0 1 1
Encoded Hamming Code (7 bits): 0 1 1 0 0 1 1
Enter bit position to introduce error (1-7, 0 = no error): 3
Transmitted Code (with error induced at pos 3): 0 1 0 0 0 1 1
Received code : 0 1 0 0 0 1 1

Checking for errors....
    Error detected at bit position: 3 -> Correcting...
corrected code : 0 1 1 0 0 1 1
Recovered Data Bits: 1 0 1 1
```

--- Block 2 ---

Enter 4 data bits (space seperated, eg: 1011) : 0110

Original Data Bits : 0110

Encoded Hamming Code (7 bits) : 1100110

Enter bit position to introduce error (1-7, 0= no error) : 6

Checking for errors ....

Error detected at bit position : 6

→ correcting ....

Corrected Code : 1100110

Recovered Data Bits : 0110

RESULT

The program to implement error detection and correction using the Hamming Code concept was successfully written in Python, executed and verified using a sample I/P data stream